

Python For and While Loops

A Beginner's Guide to Repetition in Programming

```
# Simple for loop example
for i in range(5):
    print("Hello, World!")

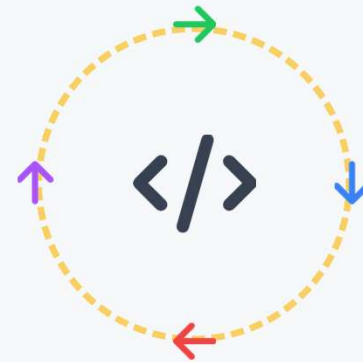
# While loop example
while i > 0:
    print(i)
    i -= 1
```



What Are Loops?

Loops are fundamental structures in programming that allow you to:

-  Automate repetitive tasks
-  Repeat actions multiple times
-  Execute code blocks until conditions are met
-  Process collections of data efficiently



Stirring a
mixture
until it
thickens



Brushing
teeth
every
morning



Adding numbers
until sum reaches
target

The For Loop Basics

What is a For Loop?

A **for loop** is used to iterate over a sequence (like a list, tuple, string, or range) and execute a block of code for each item.

- ✓ Best used when you know how many times to repeat
- ✓ Automatically handles iteration over collections

Basic Syntax

```
# Basic syntax
for item in sequence:
# Code to execute for each item
print(item)
```

Examples

Iterating over a List

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

apple banana cherry
```

Iterating over a String

```
word = "Python"
for char in word:
    print(char)

P y t h o n
```

For loops are perfect when you need to process each item in a collection!

Using Range() with For Loops

The **range()** function generates sequences of numbers for for loops.

range(stop)

Starts from 0, goes up to (but not including) stop

```
# range(5)
for i in range(5):
    print(i)
```

0 1 2 3 4

range(start, stop)

Starts from start, goes up to (but not including) stop

```
# range(2, 6)
for i in range(2, 6):
    print(i)
```

2 3 4 5

range(start, stop, step)

Starts from start, goes up to stop, incrementing by step

```
# range(0, 10, 2)
for i in range(0, 10, 2):
    print(i)
```

0 2 4 6 8

Key Points

- The **stop parameter is exclusive** - it's not included in the sequence
- Range generates numbers **on-demand**, not storing all values in memory at once
- Range works with **negative steps** for decreasing sequences

The While Loop Fundamentals

⚙️ What is a While Loop?

A **while loop** repeats execution as long as a condition remains true. It's ideal when you don't know how many iterations you'll need.

</> Basic Syntax

```
# Basic Structure
while condition:
# Code to execute
# Until condition becomes false
```

💡 Key Characteristics

- ✓ Continues until condition becomes false
- ✓ Useful for unknown iteration counts
- ✓ Requires condition update to avoid infinite loops

🎬 Example

```
# Count down from 5 to 1
count = 5
while count > 0:
    print(count)
    count -= 1
# Output: 5, 4, 3, 2, 1
```

Avoiding Infinite Loops

⚠️ What is an Infinite Loop?

A loop that runs continuously without stopping because its condition never becomes **False**.

🔍 Common Causes

- ✖ Condition variable not updated within loop
- ✖ Logic error in condition expression
- ✖ Unintended side effects modifying loop condition

🛡️ Prevention Strategies

- ✅ Always update loop control variables
- ✅ Ensure loop condition will become false
- ✅ Test with simple cases first

💣 Problematic Example

```
# This creates an infinite loop!
age = 32
while age > 18:
    print('You can vote')
# age never updated
```

✅ Fixed Example

```
# Properly updated condition
count = 1
while count <= 5:
    print(count)
    count = count + 1
# count updated, loop will end
```



Infinite



Controlled

Loop Control with Break and Continue

The **break** Statement

- ✓ Exits the loop immediately
- ✓ Terminates the entire loop
- ✓ Program continues after the loop

```
# Example with for loop
fruits = ["apple", "banana", "cherry", "orange"]
for fruit in fruits:
    if fruit == "cherry":
        # Exit the loop when cherry is found
        break
    print(fruit)
```

apple banana cherry orange → **Stops at cherry**

The **continue** Statement

- ✓ Skips the rest of the current iteration
- ✓ Continues to the next iteration
- ✓ Does not exit the loop entirely

```
# Example with for loop
numbers = [1, 2, 3, 4, 5, 6]
for num in numbers:
    if num % 2 == 0: # If even
        # Skip printing for even numbers
        continue
    print(num)
```

1 2 3 → **Skips even numbers**

 Both statements control loop execution based on conditions. Use **break** to exit early, **continue** to skip items

Choosing the Right Loop Type

For Loops

- ✓ Use when you know the **number of iterations** in advance
- ✓ Ideal for **sequences** (lists, strings, tuples)
- ✓ Automatically handles iteration variables

For Loop Example

```
# Processing a list of items
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

💡 **Best choice:** We know exactly how many items are in the list

While Loops

- ✓ Use when the number of iterations is **unknown**
- ✓ Continue until a **condition** becomes false
- ✓ Better for **user input** validation and event handling

While Loop Example

```
# Getting user input
user_input = ""
while user_input != "quit":
    user_input = input("Enter command: ")
    print(user_input)
```

💡 **Best choice:** We don't know how many inputs the user will provide

ProTip: Start with a for loop. If you don't know the number of iterations, use a while loop.